

The Ultimate SQL Mastery Guide

คู่มือเจาะลึก SQL จากศูนย์สู่ระดับ Expert ฉบับเข้มข้นที่สุดสำหรับนักพัฒนาและสายข้อมูล

Execution Order

Three-Valued Logic

Exact Date Math

Relational Joins

CTEs & Window Functions

20 Real-World Problems

Part 1: The Core Philosophy & Execution Order (กลไกสมองกลของ SQL)

ความผิดพลาด 90% ของผู้เริ่มต้นเขียน SQL มาจากความเข้าใจผิดว่าคำสั่งทำงานจากบนลงล่างตามที่เราพิมพ์ แท้จริงแล้ว SQL คือ **Declarative Language** (เราสั่งว่าจะเอาอะไร Database Optimizer จะคิดเองว่าทำอย่างไร) และมีลำดับการประมวลผลภายในดังนี้:

ลำดับการพิมพ์ (Syntax Order)	→	ลำดับการประมวลผลจริง (Logical Execution Order)	หน้าที่และการทำงานของ Database Engine
1. SELECT	→	1. FROM & JOIN	กำหนดตารางต้นทาง และทำการจับคู่แถวข้ามตาราง
2. FROM & JOIN	→	2. WHERE	กรองตัดแถวที่ไม่เข้าเงื่อนไขทิ้งก่อนเพื่อน (เร็วที่สุด)
3. WHERE	→	3. GROUP BY	ยุบรวมแถวที่มีค่าเหมือนกันเป็นกลุ่มเดียว
4. GROUP BY	→	4. HAVING	กรองผลลัพธ์ของกลุ่มที่คำนวณสถิติเสร็จแล้ว
5. HAVING	→	5. SELECT	เลือกคอลัมน์ที่จะส่งกลับ และคำนวณคอลัมน์ใหม่ (Alias)
6. ORDER BY	→	6. DISTINCT	ตัดแถวที่ข้อมูลซ้ำกันออก
7. LIMIT / OFFSET	→	7. ORDER BY	จัดเรียงลำดับแถว (ใช้ Alias จาก SELECT ได้แล้ว)
-	→	8. LIMIT	ตัดเฉพาะจำนวนแถวตามที่ระบุส่งกลับให้ผู้ใช้

💡 กฎที่ต้องจำขึ้นใจจาก Execution Order:

- ห้ามใช้ Alias ใน WHERE: เพราะ WHERE ทำงานก่อน SELECT (ระบบยังไม่รู้จักชื่อเล่นของคอลัมน์)
- สามารถใช้ Alias ใน ORDER BY ได้: เพราะ ORDER BY ทำงานหลัง SELECT แล้ว!

Part 2: Precision Filtering & Logic (การกรองข้อมูลและตรรกศาสตร์)

2.1 กฎการประเมินตssกะ (Precedence)

ลำดับความสำคัญของตัวดำเนินการ:

วงเล็บ () → NOT → AND → OR

⚠ คำเตือน: หากมี OR ปะปนกับ AND โดยไม่ใส่วงเล็บ AND จะถูกประมวลผลก่อนเสมอ ส่งผลให้ Logic ของเงื่อนไขพังทลายทันที

2.2 ความลับของ NULL (Three-Valued Logic)

ใน SQL ผลลัพธ์ทางตรรกะมี 3 ค่า: TRUE , FALSE , และ UNKNOWN

- NULL = NULL ให้ผลเป็น UNKNOWN (ไม่ใช่ TRUE!)
- WHERE จะดึงเฉพาะแถวที่เป็น TRUE เท่านั้น
- ห้ามใช้ col = NULL ให้ใช้ col IS NULL หรือ col IS NOT NULL เท่านั้น

2.3 Pattern Matching & Range Filtering

คำสั่ง	ไวยากรณ์ตัวอย่าง	พฤติกรรมและการทำงาน
LIKE	WHERE name LIKE 'ส%'	ขึ้นต้นด้วย 'ส' (เครื่องหมาย % แทนข้อความที่ตัวก็ได้ 0 ตัวขึ้นไป)
LIKE	WHERE code LIKE 'WY-___'	เครื่องหมาย _ (Underscore) แทนตัวอักษร 1 ตัวพอดี (ในตัวอย่างคือ 3 ตัวอักษร)
IN	WHERE role IN ('admin', 'staff')	เทียบเท่ากับ role = 'admin' OR role = 'staff' ทำงานเร็วกว่าและอ่านง่ายกว่า
BETWEEN	WHERE salary BETWEEN 20000 AND 35000	รวมค่าขอบทั้งสองฝั่ง (Inclusive): salary ≥ 20000 AND salary ≤ 35000

Part 3: Mastering Dates & Time (เชียนวัน-เวลา-อายุงาน)

การคำนวณเวลาคือจุดวัดกันในข้อสอบและการทำงานจริง แบ่งเป็น 4 วิธีตามระดับความแม่นยำ:

ระดับ	โจทย์สถานการณ์	ตัวอย่างคำสั่ง SQL	คำอธิบายเชิงลึก
1. ปีคงที่	คำนวณอายุงานจนถึงปี 2024	<code>(2024 - YEAR(hire_date))</code>	ใช้ฟังก์ชัน <code>YEAR()</code> ดึงเลขปีมาลบตรงๆ ง่ายและเหมาะกับข้อสอบ
2. ปีปัจจุบัน	คำนวณอายุเทียบกับปีปัจจุบันของระบบ	<code>(YEAR(CURDATE()) - YEAR(hire_date))</code>	<code>CURDATE()</code> คืนค่าวันที่ปัจจุบัน (เช่น '2026-10-10')
3. เป๊ะระดับวัน (Pro)	อายุงานเต็มปีตามกฎหมายแรงงาน	<code>TIMESTAMPDIFF(YEAR, hire_date, CURDATE())</code>	แม่นยำที่สุด! หากยังไม่ถึงวันครบรอบวันเริ่มงาน จะไม่ปัดขึ้นเป็น 1 ปี
4. นับจำนวนวัน	หาว่าทำงานมากี่วันแล้ว	<code>DATEDIFF(CURDATE(), hire_date)</code>	คืนค่าเป็นจำนวนวัน (Days) ระหว่างสองช่วงเวลา

Part 4: Aggregation & Grouping (การรวมข้อมูลและสถิติ)

 **The Golden Rule of GROUP BY:**
คอลัมน์ใดก็ตามที่อยู่ใน `SELECT` หากไม่ได้ถูกรอด้วย Aggregate Function (`SUM` , `AVG` , `COUNT` , `MIN` , `MAX`) คอลัมน์นั้น "ต้อง" ปรากฏอยู่ใน `GROUP BY` ทุกตัวเสมอ!

ศึกชิงบัลลังก์: WHERE vs HAVING ต่างกันอย่างไร?			
คุณสมบัติ	WHERE	HAVING	
จังหวะการกรอง	กรองแถวเดี่ยว ก่อน นำไปรวมกลุ่ม (Step 2)	กรองกลุ่ม หลัง จากทำ GROUP BY เสร็จแล้ว (Step 4)	
ใช้กับ SUM/AVG ได้ไหม?	❌ ห้ามใช้เด็ดขาด (Error ทันที)	✅ ใช้ได้ เช่น <code>HAVING AVG(salary) > 40000</code>	
ผลต่อประสิทธิภาพ	ช่วยลดจำนวนแถว ทำให้คำนวณรวมเร็วขึ้น	ใช้คัดเฉพาะกลุ่มผลลัพธ์สุดท้ายที่ต้องการ	

Part 5: Relational Joins & Subqueries (เชื่อมโยงมิติข้อมูล)

สรุปภาพรวม JOIN

- **INNER JOIN:** เอาเฉพาะแถวที่จับคู่กันได้ทั้งสองฝั่ง ($A \cap B$)
- **LEFT JOIN:** เอาฝั่งซ้ายครบทุกแถว ถ้าฝั่งขวาไม่มีคู่ให้ใส่ NULL
- **CROSS JOIN:** จับคู่ทุกแถว (Cartesian Product: $m \times n$ แถว)
- **SELF JOIN:** ตารางเดิม JOIN กับตัวเอง (เช่น พนักงานกับหัวหน้า)

จุดปราบเซียน: ON vs WHERE ใน LEFT JOIN

1. เงื่อนไขใน ON:

```
LEFT JOIN orders o ON c.id = o.c_id  
AND o.amt > 1000
```

→ ลูกค้าทุกคนยังอยู่ครบ! แต่เฉพาะออเดอร์ที่ > 1000 ถึงจะถูกดึงมา

2. เงื่อนไขใน WHERE:

```
LEFT JOIN orders o ON c.id = o.c_id  
WHERE o.amt > 1000
```

→ กลายสภาพเป็น INNER JOIN ทันที! ลูกค้าที่ไม่มีออเดอร์จะถูกคัดทิ้งหมด

Window Functions: อวูรลับระดับ Expert

คำนวณสถิติข้ามแถว โดยไม่ยุบแถวรวมกันเหมือน GROUP BY โครงสร้างคำสั่ง:

```
FUNCTION() OVER (PARTITION BY group_col ORDER BY sort_col)
```

ROW_NUMBER(): 1, 2, 3, 4 (ไม่สนใจคะแนนซ้ำ)

RANK(): 1, 2, 2, 4 (ซ้ำอันดับเท่ากัน แต่อันดับถัดไปกระโดด)

DENSE_RANK(): 1, 2, 2, 3 (ซ้ำอันดับเท่ากัน แต่อันดับถัดไปไม่กระโดด)

Part 9: คลังโจทย์ฝึกทำ 20 ข้อ โหด ครบ ทุกมิติ (Easy → Boss Level)

Schema อ้างอิง: employees(emp_id, emp_name, dept_id, salary, hire_date, manager_id), departments(dept_id, dept_name, location), expenses(exp_id, exp_type, amount, location, exp_date), meter_readings(id, customer_id, current_reading, reading_date)

ข้อ 1: การคำนวณคอลัมน์ใหม่และการตัดเศษทศนิยม (Easy)

พื้นฐาน

จงแสดงชื่อพนักงาน เงินเดือนปัจจุบัน และเงินเดือนสุทธิหลังหักภาษี 5% และเงินกองทุน 500 บาท โดยตั้งชื่อคอลัมน์ว่า net_salary

```
SELECT emp_name, salary, ROUND((salary * 0.95) - 500, 2) AS net_salary
FROM employees;
```

💡 ใช้ ROUND(..., 2) เพื่อควบคุมทศนิยม 2 ตำแหน่งให้ถูกต้องตามมาตรฐานการเงิน

ข้อ 2: การตรวจสอบค่าว่างที่ถูกต้องตามหลัก 3VL (Easy)

พื้นฐาน

จงแสดงรายชื่อพนักงานทุกคนที่ไม่มีหัวหน้างานโดยตรง (ระดับ CEO/Executive)

```
SELECT emp_id, emp_name
FROM employees
WHERE manager_id IS NULL;
```

💡 ห้ามเขียน manager_id = NULL เด็ดขาด เพราะจะคืนค่าเป็น UNKNOWN ทำให้ไม่มีผลลัพธ์ใดๆ แสดงออกมา

ข้อ 3: การใช้ตรรกะ OR ร่วมกับ AND และการจัดกลุ่มวงเล็บ (Medium)

ปานกลาง

จงหารายชื่อพนักงานในแผนก 1 หรือ 2 ที่มีเงินเดือนตั้งแต่ 30,000 ถึง 60,000 บาท และเริ่มงานตั้งแต่ปี 2021 เป็นต้นมา

```
SELECT emp_name, dept_id, salary, hire_date
FROM employees
WHERE (dept_id = 1 OR dept_id = 2)
AND (salary BETWEEN 30000 AND 60000)
AND hire_date ≥ '2021-01-01';
```

💡 hire_date ≥ '2021-01-01' เป็น SARGable query ที่ทำงานเร็วกว่า YEAR(hire_date) ≥ 2021

ข้อ 4: การค้นหาด้วยข้อความบางส่วนและตำแหน่งอักขระ (Medium)

ปานกลาง

จงแสดงรายการค่าใช้จ่ายที่มีคำว่า 'ซ่อม' อยู่ในประเภทค่าใช้จ่าย และเกิดขึ้นในสถานที่ที่ลงท้ายด้วยคำว่า 'เหนือ'

```
SELECT exp_id, exp_type, amount, location
FROM expenses
WHERE exp_type LIKE '%ซ่อม%'
      AND location LIKE '%เหนือ';
```

💡 %ซ่อม% จะจับคู่ทั้ง 'ค่าซ่อมบำรุง', 'ซ่อมท่อประปา', 'จัดซื้อชุดซ่อม'

ข้อ 5: คำนวณอายุงานที่เป๊ะตามวันเกิด/วันเริ่มงาน (Medium)

ปานกลาง

จงแสดงชื่อพนักงาน วันที่เริ่มงาน และอายุงานเต็มปี ณ วันปัจจุบัน เฉพาะคนที่มีอายุงานตั้งแต่ 5 ปีบริบูรณ์ขึ้นไป เรียงจากอายุงานมากไปน้อย

```
SELECT emp_name, hire_date,
       TIMESTAMPDIFF(YEAR, hire_date, CURDATE()) AS exact_years
FROM employees
WHERE TIMESTAMPDIFF(YEAR, hire_date, CURDATE()) ≥ 5
ORDER BY exact_years DESC;
```

💡 TIMESTAMPDIFF(YEAR, d1, d2) คือฟังก์ชันสากลที่คืนค่าอายุเต็มปี ไม่เกิดปัญหาปัดเศษเกินจริง

ข้อ 6: การรวมกลุ่มด้วย GROUP BY และการกรองด้วย HAVING (Medium)

ปานกลาง

จงหาชื่อแผนก จำนวนพนักงาน และเงินเดือนเฉลี่ย เฉพาะแผนกที่มีพนักงานมากกว่า 3 คน และมีเงินเดือนเฉลี่ยเกิน 35,000 บาท

```
SELECT d.dept_name,
       COUNT(e.emp_id) AS total_staff,
       ROUND(AVG(e.salary), 2) AS avg_salary
FROM departments d
JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING COUNT(e.emp_id) > 3
      AND AVG(e.salary) > 35000;
```

💡 เงื่อนไข COUNT และ AVG ต้องวางใน HAVING เท่านั้น วางใน WHERE ไม่ได้

ข้อ 7: Anti-Join Pattern (หาข้อมูลที่ไม่มีคู่สัมพันธ์) (Hard)

ขั้นสูง

จงหารายชื่อแผนกทั้งหมดที่ "ยังไม่มีพนักงานสังกัดอยู่เลยแม้แต่คนเดียว"

```
SELECT d.dept_id, d.dept_name
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL;
```

💡 นี่คือนิพจน์ Anti-Join ที่มีประสิทธิภาพสูงกว่าการเขียน `WHERE dept_id NOT IN (SELECT dept_id FROM employees)`

ข้อ 8: Conditional Aggregation ด้วย CASE WHEN (Hard)

ขั้นสูง

จงสรุปยอดเงินเดือนรวมของแต่ละแผนก โดยแยกออกเป็น 2 คอลัมน์: ยอดรวมของคนเงินเดือนสูง ($\geq 40,000$) และ ยอดรวมของคนเงินเดือนปกติ ($< 40,000$)

```
SELECT d.dept_name,
       SUM(CASE WHEN e.salary ≥ 40000 THEN e.salary ELSE 0 END) AS
high_salary_total,
       SUM(CASE WHEN e.salary < 40000 THEN e.salary ELSE 0 END) AS
normal_salary_total
FROM departments d
JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name;
```

💡 เทคนิคนี้เรียกว่า "Pivot Table in SQL" ช่วยหมุนข้อมูลแนวตั้งให้กลายเป็นแนวนอนได้ทันที

ข้อ 9: Self Join – การเชื่อมตารางเดิมเข้ากับตัวเอง (Hard)

ขั้นสูง

จงแสดงชื่อพนักงาน พร้อมแสดงชื่อหัวหน้างานของตนเอง หากใครไม่มีหัวหน้าให้แสดงข้อความ 'Top Executive' แทน

```
SELECT e.emp_name AS employee_name,  
       COALESCE(m.emp_name, 'Top Executive') AS manager_name  
FROM employees e  
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

💡 COALESCE(val, fallback) จะคืนค่าตัวแรกที่ไม่ใช่ NULL

ข้อ 10: Correlated Subquery (ซับควิรี่ที่ผูกกับตารางนอก) (Hard)

ขั้นสูง

จงหาพนักงานที่ได้รับเงินเดือน "สูงกว่าค่าเฉลี่ยเงินเดือนของแผนกตนเอง"

```
SELECT e.emp_name, e.dept_id, e.salary  
FROM employees e  
WHERE e.salary > (  
    SELECT AVG(sub.salary)  
    FROM employees sub  
    WHERE sub.dept_id = e.dept_id  
);
```

💡 sub.dept_id = e.dept_id ทำให้ Subquery คำนวณค่าเฉลี่ยเฉพาะแผนกของพนักงานแต่ละคนแบบแถวต่อแถว

ข้อ 11: การหาเงินเดือนสูงสุดอันดับที่ 2 (Second Highest Salary) (Hard)

ขั้นสูง

จงหาเงินเดือนที่สูงที่สุดเป็นอันดับ 2 โดยรองรับกรณีคนได้เงินเดือนอันดับ 1 มีหลายคนซ้ำกัน

```
SELECT MAX(salary) AS second_highest_salary  
FROM employees  
WHERE salary < (  
    SELECT MAX(salary) FROM employees  
);
```

💡 วิธีนี้ปลอดภัยที่สุดและเป็นมาตรฐาน ANSI SQL ใช้ได้กับทุกยี่ห้อ Database

ข้อ 12: DENSE_RANK() ใน Window Function (Expert)

ระดับเซียน

จงหาคนที่ได้เงินเดือนติดอันดับ Top 2 ของแต่ละแผนก

```
WITH RankedStaff AS (  
    SELECT emp_name, dept_id, salary,  
           DENSE_RANK() OVER (  
               PARTITION BY dept_id  
               ORDER BY salary DESC  
           ) AS rank_no  
    FROM employees  
)  
SELECT emp_name, dept_id, salary, rank_no  
FROM RankedStaff  
WHERE rank_no ≤ 2;
```

💡 DENSE_RANK() ป้องกันปัญหาอันดับกระโดดข้ามเมื่อมีคนได้เงินเดือนเท่ากัน

ข้อ 13: Running Total (คำนวณผลรวมสะสมข้ามแถว) (Expert)

ระดับเซียน

จงแสดงรายการค่าใช้จ่าย เรียงตามวันที่ พร้อมแสดงยอดใช้จ่ายสะสมรวมตั้งแต่ต้นปีจนถึงรายการปัจจุบัน (Running Total)

```
SELECT exp_id, exp_date, amount,  
       SUM(amount) OVER (  
           ORDER BY exp_date ASC, exp_id ASC  
       ) AS running_total  
FROM expenses;
```

💡 SUM(...) OVER (ORDER BY ...) จะบวกยอดทบขึ้นไปเรื่อยๆ ในแต่ละบรรทัด

ข้อ 14: LAG() ฟังก์ชันดึงค่าแถวก่อนหน้า (Expert)

ระดับเซียน

ในระบบจดมิเตอร์น้ำ จงดึงเลขมิเตอร์ครั้งก่อนหน้า ('previous_reading') อัตโนมัติจากประวัติในตาราง และคำนวณหน่วยใช้

```
SELECT customer_id, reading_date, current_reading,  
       LAG(current_reading, 1, 0) OVER (  
           PARTITION BY customer_id ORDER BY reading_date ASC  
       ) AS prev_reading,  
       (current_reading - LAG(current_reading, 1, 0) OVER (  
           PARTITION BY customer_id ORDER BY reading_date ASC  
       )) AS units_used  
FROM meter_readings;
```

💡 LAG(col, 1, 0) แปลว่าดึงค่าของแถวก่อนหน้า 1 แถว ถ้าไม่มีให้ใส่เลข 0

ข้อ 15: ลบข้อมูลซ้ำโดยเก็บแถวที่มี ID ล่าสุดไว้ (De-duplication Delete) (Expert)

ระดับเซียน

ตาราง logs มีข้อมูลซ้ำซ้อนในคู่ (user_id, ip_address) จงลบแถวที่ซ้ำทิ้ง โดยเก็บเฉพาะแถวที่มี id สูงสุดไว้

```
DELETE t1
FROM user_logs t1
JOIN user_logs t2 ON t1.user_id = t2.user_id
                  AND t1.ip_address = t2.ip_address
                  AND t1.id < t2.id;
```

💡 เงื่อนไข `t1.id < t2.id` ระบุว่า t1 เป็นแถวเก่า เมื่อลบ t1 แถวที่มี id มากที่สุดจะรอดปลอดภัย

ข้อ 16: การจำลอง Relational Division ด้วย NOT EXISTS (Expert)

ระดับเซียน

จงหารายชื่อลูกค้าที่เคยซื้อสินค้า "ครบทุกชิ้นที่มีในระบบ" (FOR ALL Condition)

```
SELECT c.cus_id, c.name
FROM customers c
WHERE NOT EXISTS (
    SELECT p.product_id FROM products p
    WHERE NOT EXISTS (
        SELECT 1 FROM orders o
        WHERE o.cus_id = c.cus_id AND o.product_id = p.product_id
    )
);
```

💡 Double NOT EXISTS เป็นวิธีแปลงพีชคณิตเชิงสัมพันธ์ Division ($S \div SS$) ให้เป็น SQL ที่มีประสิทธิภาพสูงสุด

ข้อ 17: อัตราการเติบโตเดือนต่อเดือน (MoM Growth Rate) (Boss Level)

บอสใหญ่

จงคำนวณยอดขายรวมรายได้ในแต่ละเดือน พร้อมหา % การเติบโตเมื่อเทียบกับเดือนก่อนหน้า

```
WITH MonthlySales AS (  
    SELECT DATE_FORMAT(order_date, '%Y-%m') AS sales_month,  
           SUM(total_price) AS current_rev  
    FROM orders  
    GROUP BY DATE_FORMAT(order_date, '%Y-%m')  
)  
SELECT sales_month, current_rev,  
       LAG(current_rev) OVER (ORDER BY sales_month) AS prev_rev,  
       ROUND(((current_rev - LAG(current_rev) OVER (ORDER BY sales_month))  
              / LAG(current_rev) OVER (ORDER BY sales_month)) * 100, 2) AS  
mom_growth_pct  
FROM MonthlySales;
```

💡 เป็นหัวใจของการทำ Financial & Business Intelligence Analysis

ข้อ 18: การ Pivot ยอดขาย 4 ไตรมาสเป็นแนวนอน (Boss Level)

บอสใหญ่

จงสรุปยอดขายแยกรายปี โดยให้ยอดขายไตรมาส Q1, Q2, Q3, Q4 แสดงเป็น 4 คอลัมน์แนวนอน

```
SELECT YEAR(order_date) AS order_year,  
       SUM(CASE WHEN QUARTER(order_date) = 1 THEN total_price ELSE 0 END) AS  
Q1_sales,  
       SUM(CASE WHEN QUARTER(order_date) = 2 THEN total_price ELSE 0 END) AS  
Q2_sales,  
       SUM(CASE WHEN QUARTER(order_date) = 3 THEN total_price ELSE 0 END) AS  
Q3_sales,  
       SUM(CASE WHEN QUARTER(order_date) = 4 THEN total_price ELSE 0 END) AS  
Q4_sales,  
       SUM(total_price) AS total_yearly_sales  
FROM orders  
GROUP BY YEAR(order_date)  
ORDER BY order_year DESC;
```

💡 ใช้ `QUARTER(date)` ร่วมกับ Conditional Aggregation สรุปงบไตรมาส

ข้อ 19: SARGable Range Optimization (Boss Level)

บอสใหญ่

จงเขียนคิวรีดึงข้อมูลบิลค่าน้ำในรอบปี 2024 ที่มีสถานะ 'PAID' ให้ทำงานเร็วที่สุดบนตารางที่มี 10 ล้านแถวและมี Index ที่คอลัมน์ created_at

```
SELECT id, customer_id, grand_total, payment_date
FROM meter_readings
WHERE payment_status = 'PAID'
      AND created_at ≥ '2024-01-01 00:00:00'
      AND created_at < '2025-01-01 00:00:00';
```

💡 ❌ อย่าใช้ `YEAR(created_at) = 2024` เพราะ Database จะต้องคำนวณฟังก์ชันทุกแถวทำให้ Index ไร้ค่า

ข้อ 20: Recursive CTE — ผังองค์กรระดับไม่จำกัด (Final Boss Level)

บอสใหญ่

จงแสดงผังองค์กรลำดับการบังคับบัญชาแบบต้นไม้ (Hierarchy Tree) เริ่มจาก CEO (id=1) ลงไปจนถึงระดับล่างสุด พร้อมแสดงระดับความลึก (Level) และเส้นทางสายงาน

```
WITH RECURSIVE OrgChart AS (
  -- 1. จุดเริ่มต้น (Anchor Member)
  SELECT emp_id, emp_name, manager_id, 1 AS lvl,
         CAST(emp_name AS CHAR(1000)) AS path
  FROM employees
  WHERE emp_id = 1

  UNION ALL

  -- 2. วนซ้ำดึงลูกน้อง (Recursive Member)
  SELECT e.emp_id, e.emp_name, e.manager_id, o.lvl + 1,
         CONCAT(o.path, ' → ', e.emp_name)
  FROM employees e
  JOIN OrgChart o ON e.manager_id = o.emp_id
)
SELECT emp_id, emp_name, lvl, path
FROM OrgChart
ORDER BY lvl, emp_id;
```

💡 `WITH RECURSIVE` เป็นฟีเจอร์ระดับสูงสุดของ SQL สำหรับจัดการข้อมูลแบบ Graph และ Tree เช่น สายการบังคับบัญชา หรือโครงสร้างโฟลเดอร์

🏆 ขอแสดงความยินดี! คุณสำเร็จหลักสูตร The Ultimate SQL Mastery

หากคุณเข้าใจและทำโจทย์ทั้ง 20 ข้อนี้ได้ด้วยตนเอง คุณไม่ได้แค่อยู่ในระดับสอบผ่านวิชาฐานข้อมูลแล้ว แต่คุณมีความรู้ทัดเทียมกับ **Senior Backend Developer / Data Engineer** ในตลาดการทำงานจริงครับ!